

Übung Anwendung von Datenbanksystemen

Musterlösung zur Prüfungsvorbereitung

inoffizielle Musterlösung
auf Basis der Aufgabenstellung von Prof. Dr.-Ing. Simon Hufnagel

Stand: 13. Juli 2026

Hinweis: Dieses Dokument enthält neben dem originalen Übungsblatt “Übung Anwendung von Datenbanksystemen” (16. März 2026) direkt im Anschluss an jede Teilaufgabe eine ausführliche Musterlösung (grau/grün hinterlegt). Die Lösungen orientieren sich am HR-Beispielschema aus der Vorlesung und den bereits im Projektordner **Übung/** vorhandenen Teillösungen. SQL-Beispiele sind für MariaDB formuliert. Bei offenen Diskussions- und Verständnisfragen wird jeweils eine begründete, aber nicht notwendigerweise einzig mögliche Antwort gegeben.

Aufgabe 1 Entity Relationship Modell

Im Rahmen dieser Übung soll ein ER-Modell für die Personalabteilung eines fiktiven, international aufgestellten Konzerns erstellt werden (Vorbereitung mit der Software *Dia*, Werkzeugpalette “ER”).

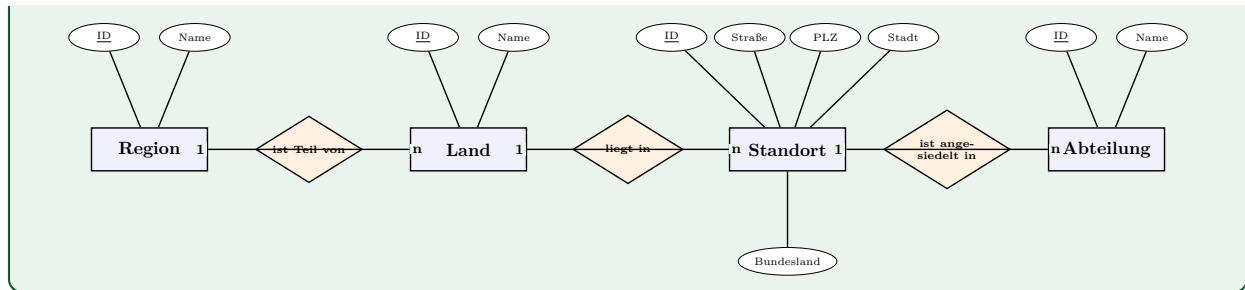
1.2 Regionen, Länder, Standorte und Abteilungen

1.2.1 Modellierung

Erstellen Sie ein ER-Modell zur Abbildung globaler Regionen, Länder, Firmenstandorte und den an den jeweiligen Standorten angesiedelten Abteilungen des Konzerns. Regionen haben eine eindeutige ID sowie einen Namen und fassen Länder zusammen. Jedes Land hat ebenfalls eine eindeutige ID sowie einen Namen und ist einer Region zugeordnet. In jedem Land kann es mehrere Firmenstandorte geben. Jeder Standort ist durch eine eindeutige ID identifizierbar und hat Adressinformationen wie Bundesland, Stadt, PLZ, Straße. Abteilungen sind wiederum jeweils einem Standort zugeordnet, durch eine ID identifiziert und haben einen Namen.

Lösung

Legende der verwendeten Chen-Notation: Rechteck = Entitätstyp, Raute = Beziehungstyp, Ellipse = Attribut (unterstrichen = Schlüsselattribut, gestrichelt = abgeleitetes Attribut), Zahlen an den Kanten = Kardinalität (1 bzw. n) aus Sicht des jeweils anderen Entitätstyps. Diese Legende gilt für alle ER-Diagramme in Aufgabe 1.



1.2.2 Diskussion

- Wie müsste das Modell angepasst werden, wenn Abteilungen über verschiedene Standorte verteilt sein können?
- Wie müsste das Modell angepasst werden, wenn statt einer expliziten Einteilung in Regionen und Länder eine beliebige geographische Struktur (z. B. Region → Land → Bundesland → Regierungsbezirk → Landkreis → Stadt) abgebildet werden können soll?

Lösung

Abteilungen über mehrere Standorte: Aus der bisherigen 1:n-Beziehung “ist angesiedelt in” müsste eine n:m-Beziehung zwischen *Standort* und *Abteilung* werden (Kardinalität auf beiden Seiten “n”). Da m:n-Beziehungen im relationalen Modell eine eigene Relation benötigen, würde später eine zusätzliche Verbindungstabelle **abteilung_standort**(*abteilung_id*, *standort_id*) entstehen. Optional könnte diese Beziehung noch um ein Attribut ergänzt werden (z. B. Anzahl der dort beschäftigten Mitarbeiter je Standort), was ohne eigene Relation gar nicht abbildbar wäre.

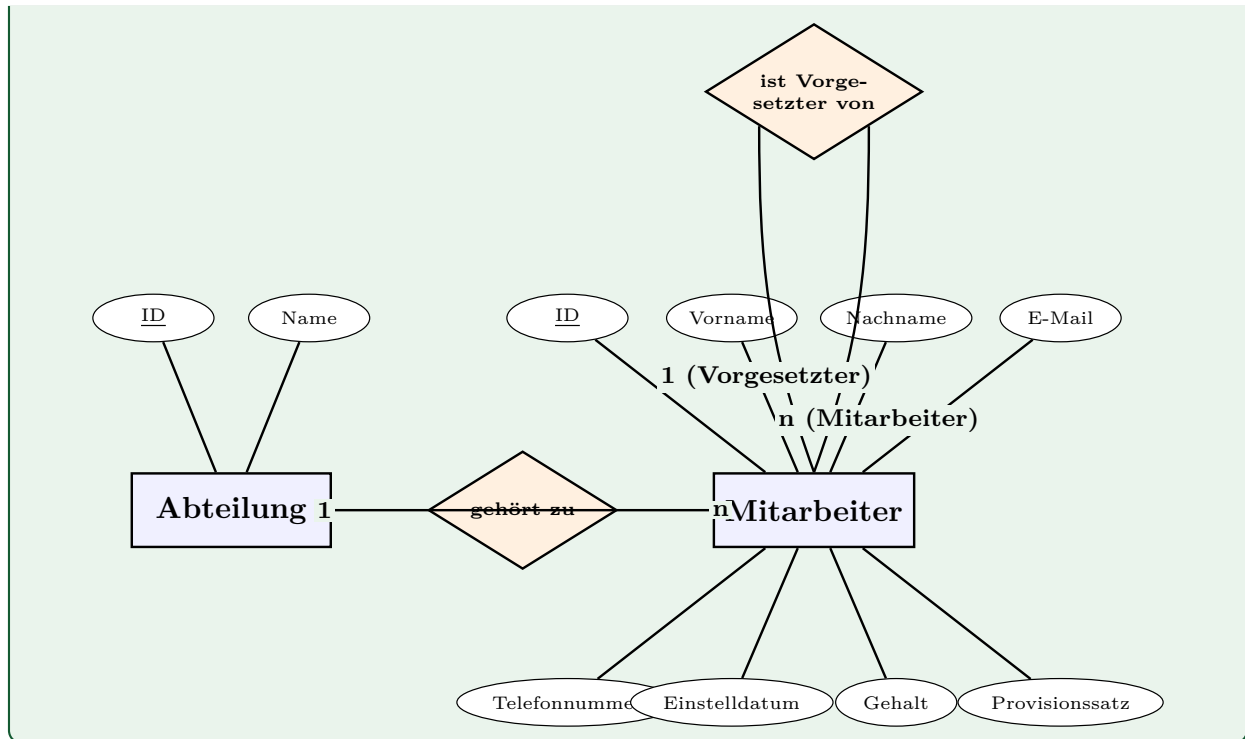
Beliebig tiefe geographische Hierarchie: Eine feste, endliche Kette von Entitätstypen (Region → Land → Bundesland → ...) kann keine *beliebig* tiefe bzw. unterschiedlich tiefe Hierarchie abbilden (nicht jedes Land hat z. B. Regierungsbezirke). Die Standardlösung ist die Einführung eines einzigen, generischen Entitätstyps “Geografische Einheit” (Attribute: ID, Name, Typ/Ebene) mit einer *rekursiven* 1:n-Beziehung “ist Teil von” auf sich selbst (analog zur später eingeführten Vorgesetzten-Beziehung bei Mitarbeitern). Damit lässt sich eine beliebig tiefe, auch pro Land unterschiedlich tiefe Baumstruktur abbilden, ohne das Schema ändern zu müssen, falls neue Hierarchieebenen hinzukommen.

1.3 Mitarbeiter

Erweitern Sie das ER-Modell um den Entitätstyp Mitarbeiter. Jeder Mitarbeiter ist einer Abteilung zugeordnet und durch eine eindeutige ID identifizierbar. Weitere Eigenschaften eines Mitarbeiters sind Vorname, Nachname, E-Mail, Telefonnummer, Einstelldatum, Gehalt und Provisionssatz. Jeder Mitarbeiter hat einen Vorgesetzten, der wiederum selbst Mitarbeiter ist. Jeder Vorgesetzte hat im Allgemeinen mehrere Mitarbeiter.

Lösung

Der neue Entitätstyp *Mitarbeiter* wird über die Beziehung “gehört zu” (1:n, ein Mitarbeiter gehört zu genau einer Abteilung, eine Abteilung hat n Mitarbeiter) an *Abteilung* angebunden. Die Vorgesetzten-Beziehung ist eine *rekursive* 1:n-Beziehung auf dem Entitätstyp *Mitarbeiter* selbst: aus Sicht der Rolle “Vorgesetzter” hat ein Mitarbeiter die Kardinalität 1, aus Sicht der Rolle “unterstellter Mitarbeiter” die Kardinalität n.



1.4 Projekte

Erweitern Sie das ER-Modell um die Möglichkeit, Projekte abzubilden. Jedes Projekt ist durch eine eindeutige ID identifizierbar und hat einen Namen. In jedem Projekt können mehrere Mitarbeiter zu jeweils unterschiedlichen zeitlichen Anteilen mitarbeiten. Mitarbeiter sollen in mehreren Projekten gleichzeitig mitarbeiten können und entweder die Projektleitung übernehmen oder als “normaler” Mitarbeiter am Projekt mitwirken.

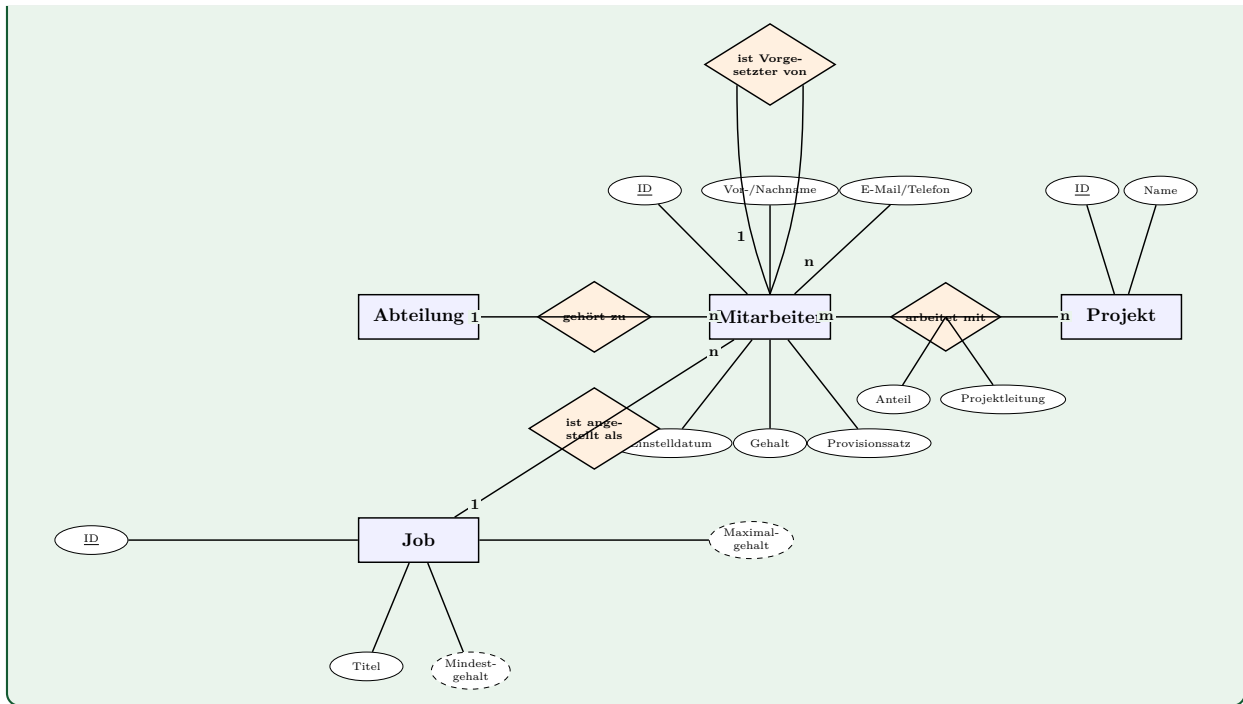
1.5 Jobs

Erweitern Sie das ER-Modell um die Abbildung von Jobs (=Jobprofile). Jeder Job ist durch eine eindeutige ID identifizierbar und hat einen Titel. Jedem Mitarbeiter ist ein Job zugeordnet. Ein Job kann mehreren Mitarbeitern zugeordnet werden. Basierend auf den Gehältern der Mitarbeiter, die im jeweiligen Job arbeiten, soll jedem Job ein Mindest- und Maximalgehalt zugeordnet werden.

Lösung (1.4 & 1.5)

Projekte: Da ein Mitarbeiter gleichzeitig an mehreren Projekten mitarbeiten kann *und* ein Projekt mehrere Mitarbeiter hat, handelt es sich um eine **n:m-Beziehung** “arbeitet mit”. Der zeitliche Anteil (**Anteil**) sowie das Merkmal, ob der Mitarbeiter die Projektleitung übernimmt (**Projektleitung**), sind Eigenschaften *des Zusammenwirkens* von Mitarbeiter und Projekt und werden daher direkt an die Beziehung selbst geheftet (in Chen-Notation dürfen n:m-Beziehungen eigene Attribute tragen).

Jobs: Zwischen *Job* und *Mitarbeiter* besteht die 1:n-Beziehung “ist angestellt als” (ein Job wird von n Mitarbeitern ausgeübt, ein Mitarbeiter übt zu einem Zeitpunkt genau einen Job aus). **Mindestgehalt** und **Maximalgehalt** sind *abgeleitete Attribute* des Entitätstyps *Job* (gestrichelt dargestellt), da sie sich aus den Gehältern der zugeordneten Mitarbeiter berechnen lassen und nicht redundanzfrei durch den Anwender gepflegt werden müssen.



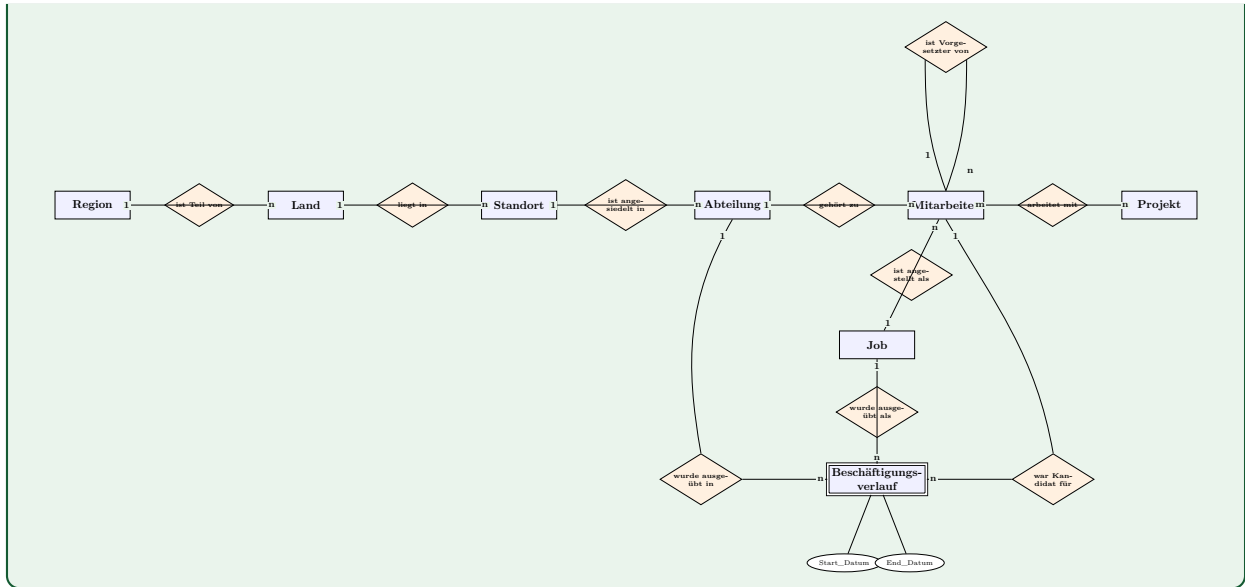
1.6 Beschäftigungsverlauf

In der Datenbank soll für jeden Mitarbeiter der jeweilige Beschäftigungsverlauf abgespeichert werden können. Dazu soll der Job und die Abteilung, in der dieser Job ausgeführt wurde, zusammen mit Start- und Enddatum gespeichert werden. Das Startdatum soll zusammen mit der ID des Mitarbeiters die Eindeutigkeit der Einträge sicherstellen. Zu berücksichtigen sind: ein Mitarbeiter kann einen Job in mehreren Abteilungen ausgeführt haben, ein Mitarbeiter kann in einer Abteilung mehrere Jobs ausgeführt haben, in einer Abteilung kann ein Job von mehreren Mitarbeitern ausgeführt werden.

Lösung

Da sich ein Beschäftigungsabschnitt gleichzeitig auf *drei* Entitätstypen bezieht (Mitarbeiter, Abteilung, Job) und keine der paarweisen Kombinationen (Mitarbeiter/Job, Mitarbeiter/Abteilung, Job/Abteilung) die Einträge eindeutig identifiziert, wird *Beschäftigungsverlauf* als eigener (schwacher) Entitätstyp mit drei 1:n-Beziehungen zu Mitarbeiter, Abteilung und Job modelliert ("war Kandidat für", "wurde ausgeübt in", "wurde ausgeübt als"). Die Eigenschaften **Start_Datum** und **End_Datum** gehören zum Beschäftigungsverlauf; die Existenz eines Eintrags hängt von allen drei Fremdbeziehungen ab, daher die doppelte Umrandung (schwacher Entitätstyp).

Dies ist das **vollständige ER-Modell**, das in Aufgabe 2 als Ausgangsbasis für die Überführung in das relationale Modell dient. Die einzelnen Attribute der bereits bekannten Entitätstypen sind hier zur besseren Übersicht ausgeblendet (siehe Diagramme zu 1.2–1.5); nur die neu eingeführten Attribute des Beschäftigungsverlaufs sind dargestellt.



Aufgabe 2 relationales Modell

Im Rahmen dieser Übung soll aus dem Entity-Relationship-Modell (siehe Lösung zu Aufgabe 1.6) ein relationales Modell erstellt werden.

2.1 Relationen der Entitätstypen

Überführen Sie alle Entitätstypen aus dem Entity-Relationship-Modell in das relationale Modell.

Lösung

Jeder Entitätstyp wird zunächst 1:1 in eine Relation überführt; die Schlüsselattribute des Entitätstyps werden zum Primärschlüssel der Relation:

- `region(region_id, region_name)`
- `country(country_id, country_name)`
- `location(location_id, street_address, postal_code, city, state_province)`
- `department(department_id, department_name)`
- `employee(employee_id, first_name, last_name, email, phone_number, hire_date, salary, commission_pct)`
- `job(job_id, job_title, min_salary, max_salary)`
- `project(project_id, project_name)`

Die abgeleiteten Attribute `min_salary`/`max_salary` werden trotz ihrer Ableitbarkeit als gewöhnliche (materialisierte) Spalten geführt, da SQL keine automatisch berechneten, immer aktuellen Attribute ohne Trigger/View kennt.

2.2 Relationen der Beziehungstypen

Überführen Sie alle Beziehungstypen des Entity-Relationship-Modells in das relationale Modell. Achten Sie insbesondere auf die korrekte Definition der Primärschlüssel.

Lösung

Grundregel: 1:n-Beziehungen benötigen *keine* eigene Relation – der Fremdschlüssel wird stattdessen als zusätzliche Spalte in die Relation auf der “n”-Seite aufgenommen. Nur n:m-Beziehungen (und mehrstellige/assoziative Beziehungen) benötigen eine eigene Verbindungsrelation.

- *ist Teil von* (1:n): `country.region_id → region.region_id`
- *liegt in* (1:n): `location.country_id → country.country_id`
- *ist angesiedelt in* (1:n): `department.location_id → location.location_id`
- *gehört zu* (1:n): `employee.department_id → department.department_id`
- *ist angestellt als* (1:n): `employee.job_id → job.job_id`
- *ist Vorgesetzter von* (1:n, rekursiv): `employee.manager_id → employee.employee_id`
- *arbeitet mit* (n:m ⇒ eigene Relation):
`employee2project(employee_id, project_id, assoc_pct, is_project_lead)`
mit `employee_id → employee.employee_id` und `project_id → project.project_id`.
Der Primärschlüssel besteht aus *beiden* Fremdschlüsseln, da genau diese Kombination die Beziehung eindeutig identifiziert.
- *Beschäftigungsverlauf* (drei 1:n-Beziehungen zu einem assoziativen Entitätstyp ⇒ eigene Relation):
`job_history(employee_id, start_date, end_date, job_id, department_id)`
mit `employee_id → employee.employee_id`, `job_id → job.job_id`, `department_id → department.department_id`. Als Primärschlüssel wird – wie in der Aufgabenstellung vorgegeben – (`employee_id`, `start_date`) verwendet (ein Mitarbeiter kann zu einem

gegebenen Startdatum immer nur einen Beschäftigungsabschnitt beginnen).

2.3 Vereinfachung des relationalen Modells

Vereinfachen Sie das entstandene relationale Modell durch geeignete Zusammenfassung der Relationen.

Lösung

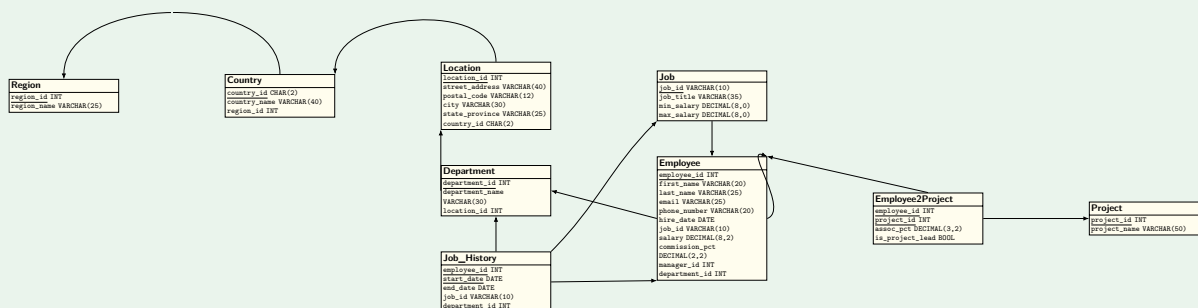
Da sämtliche 1:n-Beziehungen bereits in Schritt 2.2 direkt als Fremdschlüssel-Spalte in die referenzierende Relation integriert wurden (statt eigene Beziehungsrelationen anzulegen), ist keine weitere Zusammenfassung mehr nötig oder möglich: Eine zusätzliche Verschmelzung wäre nur bei 1:1-Beziehungen sinnvoll (hier nicht vorhanden) oder wenn fälschlicherweise auch für 1:n-Beziehungen eigene Relationen angelegt worden wären. Das vereinfachte relationale Modell besteht somit aus genau 9 Relationen: `region`, `country`, `location`, `department`, `job`, `employee`, `project`, `employee2project`, `job_history`.

2.4 Datentypen

Erweitern Sie die Relationen des vereinfachten relationalen Modells um passende Datentypen.

Lösung

Die folgende Abbildung zeigt das vollständige relationale Modell (= HR-Schema) inklusive Datentypen, Primär- (unterstrichen) und Fremdschlüsseln (Pfeile). Es ist identisch mit dem in Aufgabe 3.6/4 vorausgesetzten Schema und bildet die Grundlage für alle folgenden Übungen.



Legende: Pfeile zeigen jeweils vom Fremdschlüssel auf den referenzierten Primärschlüssel. Der Selbstbezug `employee.manager_id → employee.employee_id` ist als kleine Schleife an der *Employee*-Box angedeutet. Die vollständigen CREATE TABLE-Anweisungen inkl. aller Fremdschlüssel folgen in der Lösung zu Aufgabe 4.

Aufgabe 3 Normalisierung

3.1 Normalformen

Gegeben sei das folgende relationale Modell zur Verwaltung der offenen Rechnungen eines Onlineshops inkl. Beispieldaten:

<u>RNr</u>	KDNr	Name	Wohnort	Pos	Datum	Betrag
1	1	Müller	München	3	01.11.2021	60
2	1	Müller	München	2	23.05.2021	90
3	2	Huber	Nürnberg	2	09.03.2022	90
4	2	Huber	Nürnberg	2	14.02.2022	70
5	3	Meier	Augsburg	3	20.06.2022	110
6	4	Meier	München	4	07.04.2022	90

3.1.1 2. Normalform

Erläutern Sie, warum nur Relationen mit mehrwertigen (zusammengesetzten) Schlüsselkandidaten die 2. NF verletzen können.

Lösung

Die 2. NF verlangt, dass jedes Nicht-Schlüsselattribut *voll* funktional (und nicht nur partiell) von *jedem* Schlüsselkandidaten abhängt. Eine partielle Abhängigkeit setzt per Definition voraus, dass ein Nicht-Schlüsselattribut bereits von einer *echten Teilmenge* eines Schlüsselkandidaten bestimmt wird. Besteht ein Schlüsselkandidat jedoch nur aus einem einzigen Attribut, besitzt er überhaupt keine nichttriviale echte Teilmenge (außer der leeren Menge, die per Konvention nichts funktional bestimmen kann) – eine partielle Abhängigkeit ist damit strukturell unmöglich. Erst wenn ein Schlüsselkandidat aus *mehreren* Attributen besteht, kann ein Nicht-Schlüsselattribut von einer echten (nichtleeren) Teilmenge dieses zusammengesetzten Schlüssels abhängen, ohne von allen Schlüsselattributen abzuhängen – nur dann ist eine 2. NF-Verletzung überhaupt möglich.

3.2 funktionale Abhängigkeiten

Geben Sie für das gegebene relationale Modell alle funktionalen Abhängigkeiten an.

Lösung

Der (einzige) Schlüsselkandidat der Relation ist **RNr** (jede Rechnungsnummer kommt in den Beispieldaten genau einmal vor). Es ergeben sich folgende funktionalen Abhängigkeiten:

$\text{RNr} \rightarrow \text{KDNr}, \text{Pos}, \text{Datum}, \text{Betrag}$ (Schlüssel $\rightarrow$ alle übrigen Attribute)

$\text{KDNr} \rightarrow \text{Name}, \text{Wohnort}$ (Kundennummer bestimmt Name und Wohnort)

Aus $\text{RNr} \rightarrow \text{KDNr}$ und $\text{KDNr} \rightarrow \text{Name}, \text{Wohnort}$ folgt transitiv: $\text{RNr} \rightarrow \text{Name}, \text{Wohnort}$.

3.3 3. Normalform

Erläutern Sie, inwiefern das gegebene relationale Modell nicht der 3. NF entspricht. Zeigen Sie beispielhaft, welche Anomalien auftreten können. Überführen Sie das Modell in die dritte Normalform.

Lösung

Verletzung: Die Attribute **Name** und **Wohnort** hängen nicht direkt vom Schlüssel **RNr** ab, sondern nur *transitiv* über das Nicht-Schlüsselattribut **KDNr** ($RNr \rightarrow KDNr \rightarrow Name, Wohnort$). Damit liegt eine transitive Abhängigkeit eines Nicht-Schlüsselattributs von einem anderen Nicht-Schlüsselattribut vor – ein klassischer Verstoß gegen die 3. NF.

Anomalien (am Beispiel):

- *Änderungsanomalie:* Zieht Herr Müller (KDNr 1) um, muss sein Wohnort in *beiden* Zeilen (RNr 1 und 2) geändert werden – vergisst man eine Zeile, entsteht ein inkonsistenter Datenbestand.
- *Einfügeanomalie:* Ein neuer Kunde kann nicht gespeichert werden, solange er noch keine Rechnung hat, da Name/Wohnort nur über eine Rechnungszeile erfasst werden.
- *Löschanomalie:* Löscht man die einzige Rechnung eines Kunden (z.B. RNr 5 für Meier/Augsburg), gehen dessen Stammdaten (Name, Wohnort) vollständig und unwiederbringlich verloren.

Überführung in die 3. NF durch Auslagerung der transitiv abhängigen Attribute in eine eigene Relation:

- Kunde(KDNr, Name, Wohnort)
- Rechnung(RNr, KDNr, Pos, Datum, Betrag) mit $KDNr \rightarrow Kunde.KDNr$

3.4 Boyce-Codd Normalform

Erläutern Sie, ob das in die 3. NF überführte relationale Modell der Boyce-Codd-Normalform entspricht, und bringen Sie das Modell ggf. in die BCNF.

Lösung

Die BCNF verlangt, dass für *jede* nichttriviale funktionale Abhängigkeit $X \rightarrow Y$ die linke Seite X ein Superschlüssel ist. In Kunde(KDNr, Name, Wohnort) ist $KDNr \rightarrow Name, Wohnort$ die einzige nichttriviale FD, und KDNr ist bereits Schlüssel $\Rightarrow$ BCNF erfüllt. In Rechnung(RNr, KDNr, Pos, Datum, Betrag) ist $RNr \rightarrow KDNr, Pos, Datum, Betrag$ die einzige nichttriviale FD, und RNr ist Schlüssel $\Rightarrow$ ebenfalls BCNF erfüllt (unter der plausiblen Annahme, dass KDNr nicht umgekehrt RNr bestimmt, da ein Kunde mehrere Rechnungen haben kann). **Die Zerlegung aus 3.3 ist somit bereits in BCNF**, eine weitere Zerlegung ist nicht nötig.

3.5 Normalform des Hochschul-Schemas

Bestimmen Sie, in welcher Normalform das Hochschul-Schema (Dozent, Vorlesung, Kapitel, Studierende, hört, Raumbellegung, Prüfung, schreibt) vorliegt. Begründen Sie Ihre Antwort.

Lösung

Eine relationenweise Analyse (Primärschlüssel jeweils unterstrichen laut ER-Diagramm):

- Dozent(Personalnummer, Name, Telefonnummer, E-Mail), Studierende(Matrikelnummer, Name, Studiengang), Vorlesung(Modulnummer, Name, Personalnummer): jeweils **einwertiger** Schlüssel $\Rightarrow$ 2. NF automatisch erfüllt (siehe 3.1.1); keine weiteren Nicht-Schlüsselattribute hängen voneinander ab $\Rightarrow$ auch 3. NF/BCNF erfüllt.
- Kapitel(Modulnummer, Überschrift, Inhalt): zusammengesetzter Schlüssel; Inhalt hängt von *beiden* Teilen ab (derselbe Kapiteltitel hat in unterschiedlichen Modulen unterschiedlichen Inhalt) $\Rightarrow$ keine partielle Abhängigkeit, 2. NF erfüllt; da nur ein Nicht-Schlüsselattribut existiert, ist auch keine transitive Abhängigkeit möglich $\Rightarrow$ 3. NF/BCNF erfüllt.

- **hört**(Matrikelnummer, Modulnummer, Semester): besteht nur aus Schlüsselattributen, es gibt keine Nicht-Schlüsselattribute $\Rightarrow$ trivial in BCNF (keine FD kann verletzt werden).
- **Raumbelegung**(Raumnummer, Wochentag, Uhrzeit, Semester, Modulnummer): das einzige Nicht-Schlüsselattribut Modulnummer hängt von der *vollständigen* Schlüsselkombination ab (welches Modul in einem bestimmten Raum an einem Wochentag/einer Uhrzeit in einem Semester stattfindet) – keine der vier Teilmengen bestimmt Modulnummer allein $\Rightarrow$ 2.NF erfüllt; nur ein Nicht-Schlüsselattribut $\Rightarrow$ keine Transitivität möglich $\Rightarrow$ 3.NF/BCNF erfüllt.
- **Prüfung**(ID, Aufsicht, Zeitpunkt, Raum, Semester): einwertiger Schlüssel $\Rightarrow$ analog zu Dozent/Studierende in BCNF.
- **schreibt**(Modulnummer, PrüfungsID, Matrikelnummer, Note): dass Modulnummer trotz eindeutiger PrüfungsID explizit Teil des Schlüssels ist, deutet darauf hin, dass eine Prüfung (z. B. eine gemeinsame Klausur) mehrere Module gleichzeitig abprüfen kann; Note hängt dann von der vollständigen Kombination aus Modul, Prüfung *und* Studierenden ab $\Rightarrow$ keine partielle, mangels weiterer Nicht-Schlüsselattribute auch keine transitive Abhängigkeit $\Rightarrow$ BCNF erfüllt.

Ergebnis: Unter den (realistischen) Annahmen, dass keine der oben genannten Attributkombinationen noch durch eine echte Teilmenge des jeweiligen Schlüssels bestimmt wird, befindet sich das **gesamte Hochschul-Schema in der Boyce-Codd-Normalform (BCNF)**.

3.6 Normalform des HR-Schemas

Bestimmen Sie, in welcher Normalform das HR-Schema vorliegt. Begründen Sie Ihre Antwort.

Lösung

Alle neun Relationen des in Aufgabe 2.4 hergeleiteten HR-Schemas besitzen entweder einen einwertigen Primärschlüssel (**region**, **country**, **location**, **department**, **job**, **employee**, **project**) oder einen zusammengesetzten Schlüssel, bei dem die Nicht-Schlüsselattribute nachweislich von der *vollständigen* Schlüsselkombination abhängen (**job_history**: Job und Abteilung eines Beschäftigungsabschnitts hängen vom Mitarbeiter *und* vom Startdatum ab, nicht von einem der beiden Teile allein; **employee2project**: **assoc_pct/is_project_lead** hängen von der konkreten Mitarbeiter-Projekt-Kombination ab). Es liegen keine weiteren funktionalen Abhängigkeiten vor, deren linke Seite kein Superschlüssel wäre. **Das HR-Schema befindet sich damit – wie in Aufgabe 2 hergeleitet – in der BCNF.**

Zwei häufig diskutierte Sonderfälle sollten dabei aber angesprochen werden:

1. **location.postal_code**: In vielen Ländern legt die Postleitzahl Stadt und ggf. Bundesland eindeutig fest (**postal_code** $\rightarrow$ **city**, **state_province**), was wegen **location_id** $\rightarrow$ **postal_code** $\rightarrow$ **city** formal eine transitive Abhängigkeit und damit einen 3.NF-Verstoß bedeuten würde. Da das Schema jedoch international unterschiedliche, teils fehlende (NULL) Postleitzahlen abbildet (z. B. London, Rom, Hiroshima, Singapur ohne PLZ), gilt diese funktionale Abhängigkeit *nicht global* über den gesamten Datenbestand – sie wird hier daher nicht als Normalform-Verstoß gewertet, sollte in der Diskussion aber als Modellierungs-Trade-off erwähnt werden.
2. **job.min_salary/max_salary**: Diese Werte sind laut Aufgabenstellung 1.5 aus **employee.salary** *abgeleitet*. Das ist zwar *innerhalb* der Relation **job** kein BCNF-Verstoß (der Schlüssel **job_id** bleibt Determinante), stellt aber eine *relationenübergreifende* Redundanz/Ableitung dar, die bei Gehaltsanpassungen manuell konsistent gehalten werden muss (klassisches Beispiel für “denormalisierte”, abgeleitete Daten außerhalb des eigentlichen Normalformen-Konzepts).

Aufgabe 4 SQL Data Definition Language – Teil 1

Ziel: Im Rahmen dieser Übung sollen die grundlegenden SQL-Befehle für die Implementierung des HR-Schemas mit Hilfe eines MariaDB-DBS erstellt werden.

4.2 Schema-Erstellung

Erstellen Sie die notwendigen SQL-Befehle, um in der Datenbasis ein neues Schema mit dem Namen “hr” zu erstellen (Datei HR_setupDB.sql, beliebig oft ausführbar).

Lösung

```
DROP DATABASE IF EXISTS hr;
CREATE DATABASE hr;
USE hr;
```

Das DROP DATABASE IF EXISTS stellt sicher, dass das Skript beliebig oft ausgeführt werden kann und das Schema bei jedem Lauf komplett neu aufgebaut wird.

4.3 Relationen, Attribute und Datentypen

Erstellen Sie die notwendigen SQL-Befehle, um innerhalb des “hr”-Schemas alle Relationen mit den jeweiligen Attributen und Datentypen (inkl. Primärschlüssel) zu erstellen.

Lösung

```
CREATE TABLE region (
    region_id    INT NOT NULL,
    region_name  VARCHAR(25),
    PRIMARY KEY (region_id)
);

CREATE TABLE country (
    country_id   CHAR(2) NOT NULL,
    country_name VARCHAR(40),
    region_id    INT NOT NULL,
    PRIMARY KEY (country_id)
);

CREATE TABLE location (
    location_id   INT NOT NULL AUTO_INCREMENT,
    street_address VARCHAR(40),
    postal_code   VARCHAR(12),
    city          VARCHAR(30) NOT NULL,
    state_province VARCHAR(25),
    country_id    CHAR(2) NOT NULL,
    PRIMARY KEY (location_id)
);

CREATE TABLE department (
    department_id INT NOT NULL,
    department_name VARCHAR(30) NOT NULL,
    location_id   INT,
    PRIMARY KEY (department_id)
);

CREATE TABLE job (
    job_id    VARCHAR(10) NOT NULL,
```

```

    job_title    VARCHAR(35) NOT NULL,
    min_salary   DECIMAL(8,0),
    max_salary   DECIMAL(8,0),
    PRIMARY KEY (job_id)
);

CREATE TABLE employee (
    employee_id   INT NOT NULL,
    first_name    VARCHAR(20),
    last_name     VARCHAR(25) NOT NULL,
    email         VARCHAR(25) NOT NULL,
    phone_number  VARCHAR(20),
    hire_date     DATE NOT NULL,
    job_id        VARCHAR(10) NOT NULL,
    salary        DECIMAL(8,2) NOT NULL,
    commission_pct DECIMAL(2,2),
    manager_id    INT,
    department_id INT,
    PRIMARY KEY (employee_id)
);

CREATE TABLE job_history (
    employee_id   INT NOT NULL,
    start_date    DATE NOT NULL,
    end_date      DATE NOT NULL,
    job_id        VARCHAR(10) NOT NULL,
    department_id INT NOT NULL,
    PRIMARY KEY (employee_id, start_date)
);

CREATE TABLE project (
    project_id    INT NOT NULL,
    project_name  VARCHAR(50),
    PRIMARY KEY (project_id)
);

CREATE TABLE employee2project (
    employee_id   INT NOT NULL,
    project_id    INT NOT NULL,
    assoc_pct     DECIMAL(3,2),
    is_project_lead BOOL,
    PRIMARY KEY (employee_id, project_id)
);

```

Optionalität der Attribute: Attribute, die laut Aufgabenstellung immer vorhanden sein müssen (z.B. Namen, Datumswerte, E-Mail), sind NOT NULL; optionale Attribute (z.B. commission_pct, manager_id beim CEO, department_id vor Zuordnung) bleiben nullable.

4.4 Fremdschlüsselbeziehungen

4.4.1 Implementierung

Ergänzen Sie die notwendigen SQL-Befehle, um die Fremdschlüsselbeziehungen zwischen den Relationen unter Berücksichtigung der vorgegebenen Referenzierungsoptionen herzustellen.

Lösung

```

ALTER TABLE country
  ADD FOREIGN KEY (region_id) REFERENCES region(region_id)
  ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE location
  ADD FOREIGN KEY (country_id) REFERENCES country(country_id)
  ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE department
  ADD FOREIGN KEY (location_id) REFERENCES location(location_id)
  ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE employee
  ADD FOREIGN KEY (job_id) REFERENCES job(job_id)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  ADD FOREIGN KEY (department_id) REFERENCES department(department_id)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  ADD FOREIGN KEY (manager_id) REFERENCES employee(employee_id)
  ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE job_history
  ADD FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
  ON UPDATE CASCADE ON DELETE CASCADE,
  ADD FOREIGN KEY (job_id) REFERENCES job(job_id)
  ON UPDATE CASCADE ON DELETE CASCADE,
  ADD FOREIGN KEY (department_id) REFERENCES department(department_id)
  ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE employee2project
  ADD FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
  ON UPDATE CASCADE ON DELETE CASCADE,
  ADD FOREIGN KEY (project_id) REFERENCES project(project_id)
  ON UPDATE CASCADE ON DELETE CASCADE;

```

4.4.2 Verständnisfragen

Erläutern Sie die Auswirkungen folgender Änderungen am Datenbestand unter Berücksichtigung der o. g. Referenzierungsoptionen.

Lösung

- **Wirkung von RESTRICT bei employee(department_id):** Solange mindestens ein Mitarbeiter einer Abteilung zugeordnet ist, verhindert RESTRICT das Löschen dieser Abteilung – die DELETE-Anweisung auf department wird mit einem Fremdschlüsselfehler abgelehnt. Die Abteilung darf erst gelöscht werden, nachdem alle zugehörigen Mitarbeiter umgezogen oder entfernt wurden.
- **location_id einer bestehenden Location wird aktualisiert:** Da department.location_id mit ON UPDATE CASCADE definiert ist, wird der neue Wert automatisch in allen referenzierenden department-Zeilen übernommen – die Änderung erfolgt transparent, ohne Fehlermeldung.
- **Löschen einer Location mit zugeordneten Departments:** Die DELETE-Anweisung wird abgelehnt (Fremdschlüsselverletzung), da location(location_id) →

department(location_id) mit ON DELETE RESTRICT definiert ist. Die Departments müssten zuerst gelöscht oder umgezogen werden.

- **Löschen eines Mitarbeiters aus der Employee-Relation:**

- *Der Mitarbeiter ist Manager:* Solange andere Mitarbeiter ihn über `manager_id` referenzieren, verhindert RESTRICT das Löschen – die unterstellten Mitarbeiter müssten zuerst einem anderen Manager zugeordnet werden.

- *Der Mitarbeiter ist kein Manager:* Das Löschen ist erlaubt. Da `job_history(employee_id)` und `employee2project(employee_id)` mit ON DELETE CASCADE definiert sind, werden dabei automatisch auch alle Beschäftigungshistorien- und Projektzuordnungs-Einträge dieses Mitarbeiters mitgelöscht.

- **Alle Lösch-Optionen wären CASCADE: Was passiert beim Löschen einer Region?** Es käme zu einer weitreichenden Kettenreaktion: Region → alle zugehörigen Countries werden gelöscht → deren Locations werden gelöscht → deren Departments werden gelöscht → alle Employees dieser Departments werden gelöscht (inkl. rekursiv aller ihrer Mitarbeiter über `manager_id`) → deren `job_history`- und `employee2project`-Einträge werden gelöscht. Eine einzige DELETE-Anweisung auf einer einzelnen Region könnte damit einen Großteil der Datenbank löschen. Genau deshalb verwendet das vorgegebene Schema bewusst RESTRICT für die “strukturellen” Fremdschlüssel der geografischen/organisatorischen Hierarchie (und für `manager_id`) und nur für die reinen “Historien-/Detailtabellen” (`job_history`, `employee2project`) CASCADE.

Aufgabe 5 Constraints

Im Rahmen dieser Aufgabe soll das HR-Schema um weitere Regeln zur automatischen Sicherstellung der Datenintegrität erweitert werden (Ergänzungen in HR_setupDB.sql).

5.1 Arithmetische Constraints

- `min_salary` in der `job`-Relation soll positiv sein
- `max_salary` in der `job`-Relation soll mindestens so groß wie `min_salary` sein
- `salary` und `commission_pct` in der `employee`-Relation sollen positiv sein
- `end_date` in der `job_history`-Relation darf nicht vor dem `start_date` liegen
- `assoc_pct` in der `employee2project`-Relation muss größer als 0 und kleiner gleich 1 sein

5.2 Reguläre Ausdrücke

- `street_address` in der `location`-Relation soll an einer beliebigen Stelle mindestens eine Ziffer enthalten (Hausnummer)
- die `email`-Adresse in der `employee`-Relation soll dem Muster `[a-zA-Z][a-zA-Z0-9]*@[a-zA-Z]+\.[a-z]{2,4}` entsprechen

Lösung (5.1 & 5.2)

```
ALTER TABLE job
  ADD CONSTRAINT MinSalaryPositive CHECK (min_salary > 0),
  ADD CONSTRAINT MaxSalaryGreaterThanMinSalary CHECK (max_salary >=
    min_salary);

ALTER TABLE employee
  ADD CONSTRAINT SalaryPositive CHECK (salary > 0),
  ADD CONSTRAINT CommissionPctPositive CHECK (commission_pct > 0),
  ADD CONSTRAINT EMailPattern
    CHECK (email RLIKE '^ [a-zA-Z][a-zA-Z0-9]*@[a-zA-Z]+\.[a-z]{2,4}
    $');

ALTER TABLE job_history
  ADD CONSTRAINT EndDateAfterStartDate CHECK (end_date > start_date);

ALTER TABLE employee2project
  ADD CONSTRAINT AssocPctRange CHECK (assoc_pct > 0 AND assoc_pct <=
    1);

ALTER TABLE location
  ADD CONSTRAINT AtLeastOneDigitInStreetAddress CHECK (street_address
    RLIKE '[0-9]');
```

Hinweis zum regulären Ausdruck der E-Mail-Adresse: `^` und `$` verankern den Ausdruck an Anfang/Ende; `[a-zA-Z]` erzwingt einen Buchstaben als erstes Zeichen; `[a-zA-Z0-9]*` erlaubt beliebig viele weitere alphanumerische Zeichen vor dem `@`; `[a-zA-Z]+\.` verlangt mindestens einen Buchstaben gefolgt von einem Punkt; `[a-z]{2,4}` erzwingt die 2–4-stellige Top-Level-Domain in Kleinbuchstaben.

5.3 Komplexe Constraints

Identifizieren Sie weitere sinnvolle Constraints, die über die einfache Prüfung der Attributwerte eines neuen Tupels hinausgehen und z. B. bereits vorhandene Tupel mit einbeziehen.

Lösung

Einfache CHECK-Constraints in MariaDB können nur Attribute *derselben* Zeile prüfen – sie können weder andere Zeilen derselben Tabelle noch andere Tabellen einbeziehen. Für zeilenübergreifende bzw. tabellenübergreifende Geschäftsregeln werden daher TRIGGER benötigt. Sinnvolle Kandidaten wären u. a.:

1. **Ein Mitarbeiter darf nicht sein eigener Vorgesetzter sein** – lässt sich noch als einfacher CHECK auf der eigenen Zeile formulieren:

```
ALTER TABLE employee
  ADD CONSTRAINT NotOwnManager CHECK (employee_id <> manager_id)
;
```

2. **Das Gehalt eines Mitarbeiters muss innerhalb des Gehaltsrahmens (min_salary/max_salary) seines Jobs liegen** – benötigt einen Blick in die job-Tabelle:

```
DELIMITER $$
CREATE TRIGGER trg_salary_in_range
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
  DECLARE v_min DECIMAL(8,0);
  DECLARE v_max DECIMAL(8,0);
  SELECT min_salary, max_salary INTO v_min, v_max
    FROM job WHERE job_id = NEW.job_id;
  IF NEW.salary < v_min OR NEW.salary > v_max THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Gehalt liegt ausserhalb des
        Gehaltsrahmens des Jobs';
  END IF;
END$$
DELIMITER ;
```

3. **Die Summe der assoc_pct-Werte eines Mitarbeiters darf 1 (100 %) nicht überschreiten** – erfordert eine Aggregation über die bereits vorhandenen Zeilen desselben Mitarbeiters in employee2project (in den Beispieldaten aus Aufgabe 8.4.1 bewusst *nicht* durchgesetzt, um überlastete Mitarbeiter demonstrieren zu können):

```
DELIMITER $$
CREATE TRIGGER trg_assoc_pct_sum
BEFORE INSERT ON employee2project
FOR EACH ROW
BEGIN
  DECLARE v_sum DECIMAL(5,2);
  SELECT IFNULL(SUM(assoc_pct),0) INTO v_sum
    FROM employee2project WHERE employee_id = NEW.employee_id;
  IF v_sum + NEW.assoc_pct > 1 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Mitarbeiter waere zu mehr als 100%
        zugeordnet';
  END IF;
END$$
DELIMITER ;
```

4. **Zeiträume in job_history dürfen sich für denselben Mitarbeiter nicht überlappen und das start_date eines Beschäftigungsabschnitts darf nicht vor dem hire_date des Mitarbeiters liegen** – beides erfordert ebenfalls den Vergleich mit bere-

its vorhandenen Zeilen (`job_history` bzw. `employee`) und würde analog per `TRIGGER` umgesetzt.

Aufgabe 6 Data Manipulation Language

6.1 Einfügen von Daten

Erweitern Sie die bereitgestellte Datei “Aufgabe6_Datenbestand.sql” um geeignete SQL-Statements zum Einfügen von Regionen, den fehlenden Ländern (Italien, Japan, USA, Kanada, Ägypten) sowie den folgenden Projektzuordnungen: Julia Dellinger → Lean Management (70 %), Patrick Sully → US Sales Campaign (100 %, Projektleitung), Matthew Weiss → Europe Sales Campaign (20 %).

Lösung

Die bereitgestellte Datei enthält bereits Standorte, Abteilungen, Jobs, Mitarbeiter, Beschäftigungshistorien und Projekte – es fehlen lediglich die Regionen, ein Teil der Länder sowie die drei neuen Projektzuordnungen:

```
-- Regionen
INSERT INTO region VALUES (1, 'Europe');
INSERT INTO region VALUES (2, 'Americas');
INSERT INTO region VALUES (3, 'Asia');
INSERT INTO region VALUES (4, 'Middle East and Africa');

-- fehlende Laender
INSERT INTO country VALUES ('IT', 'Italy', 1);
INSERT INTO country VALUES ('JP', 'Japan', 3);
INSERT INTO country VALUES ('US', 'United States of America', 2);
INSERT INTO country VALUES ('CA', 'Canada', 2);
INSERT INTO country VALUES ('EG', 'Egypt', 4);

-- Projektzuordnungen
-- Julia Dellinger (employee_id 186) -> Lean Management (project_id 102)
INSERT INTO employee2project VALUES (186, 102, 0.70, FALSE);
-- Patrick Sully (employee_id 157) -> US Sales Campaign (project_id 100), Projektleiter
INSERT INTO employee2project VALUES (157, 100, 1.00, TRUE);
-- Matthew Weiss (employee_id 120) -> Europe Sales Campaign (project_id 101)
INSERT INTO employee2project VALUES (120, 101, 0.20, FALSE);
```

6.2 Aktualisieren von Daten

- Die neue Telefonnummer von Michael Rogers lautet 650.127.3418
- Der Mindest-Verdienst aller Jobs soll bei Bedarf auf 4000 angehoben werden
- Alle Gehälter sollen jeweils um 3% angehoben werden

Lösung

```
UPDATE employee
  SET phone_number = '650.127.3418'
  WHERE first_name = 'Michael' AND last_name = 'Rogers';

UPDATE job
  SET min_salary = 4000
  WHERE min_salary < 4000;

UPDATE employee
  SET salary = salary * 1.03;
```

6.3 Löschen von Daten

- Alle Mitarbeiter, die zu 25 % oder weniger zu einem Projekt zugeordnet sind, sollen aus dem jeweiligen Projekt entfernt werden
- Der Mitarbeiter Jonathon Taylor verlässt das Unternehmen. Vergleichen Sie den Inhalt der Relation `job_history` vor und nach der Ausführung des `DELETE`-Statements.

Lösung

```
DELETE FROM employee2project WHERE assoc_pct <= 0.25;

-- vorher: Inhalt der Job_History von Jonathon Taylor (employee_id 176)
-- prüfen
SELECT * FROM job_history WHERE employee_id = 176;
-- liefert 2 Zeilen: SA_REP (1998-03-24 bis 1998-12-31) und SA_MAN
-- (1999-01-01 bis 1999-12-31)

DELETE FROM employee WHERE first_name = 'Jonathon' AND last_name = '
    Taylor';

-- nachher:
SELECT * FROM job_history WHERE employee_id = 176;
-- liefert 0 Zeilen
```

Da `job_history(employee_id)` mit `ON DELETE CASCADE` auf `employee(employee_id)` verweist, werden beim Löschen des Mitarbeiters automatisch *beide* zugehörigen Beschäftigungshistorien-Einträge mitgelöscht – ohne dass ein explizites `DELETE` auf `job_history` nötig wäre.

6.4 Testen der Constraints

Testen Sie die Check-Constraints aus Aufgabe 5, indem Sie jeweils ein `INSERT`-Statement für einen gültigen Datensatz und einen Datensatz, der das jeweilige Constraint verletzt, erstellen und ausführen.

Lösung

```
-- MinSalaryPositive: gueltig vs. verletzt
INSERT INTO job VALUES ('TEST_OK1', 'Testjob', 3000, 6000); -- OK
INSERT INTO job VALUES ('TEST_FAIL1', 'Testjob', -100, 6000); -- Fehler
: min_salary <= 0

-- MaxSalaryGreaterThanMinSalary
INSERT INTO job VALUES ('TEST_OK2', 'Testjob', 3000, 6000); -- OK
INSERT INTO job VALUES ('TEST_FAIL2', 'Testjob', 5000, 3000); -- Fehler
: max_salary < min_salary

-- SalaryPositive / CommissionPctPositive (employee)
INSERT INTO employee VALUES (900, 'Test', 'Ok', 'TOK@example.com', '
    000.000.0000',
    STR_TO_DATE('01-JAN-2020', '%d-%M-%Y'), 'IT_PROG', 4000, 0.1, NULL,
    60); -- OK
INSERT INTO employee VALUES (901, 'Test', 'Fail', 'TFAIL@example.com',
    '000.000.0000',
    STR_TO_DATE('01-JAN-2020', '%d-%M-%Y'), 'IT_PROG', -500, 0.1, NULL,
    60); -- Fehler: salary <= 0

-- EndDateAfterStartDate (job_history)
```

```
INSERT INTO job_history VALUES (100, STR_TO_DATE('01-JAN-2000', '%d-%M-%Y'),
    STR_TO_DATE('01-JAN-2001', '%d-%M-%Y'), 'IT_PROG', 60); -- OK
INSERT INTO job_history VALUES (100, STR_TO_DATE('01-JAN-2001', '%d-%M-%Y'),
    STR_TO_DATE('01-JAN-2000', '%d-%M-%Y'), 'IT_PROG', 60); -- Fehler:
    end_date < start_date

-- AssocPctRange (employee2project)
INSERT INTO employee2project VALUES (100, 102, 0.5, FALSE); -- OK
INSERT INTO employee2project VALUES (100, 102, 1.5, FALSE); -- Fehler:
    assoc_pct > 1

-- AtLeastOneDigitInStreetAddress (location)
INSERT INTO location (street_address, city, country_id) VALUES ('
    Musterstr. 12', 'Testort', 'DE'); -- OK
INSERT INTO location (street_address, city, country_id) VALUES ('
    Musterstrasse ohne Hausnummer', 'Testort', 'DE'); -- Fehler: keine
    Ziffer

-- EMailPattern (employee)
INSERT INTO employee VALUES (902, 'Test', 'Mail', 'tmail@example.com',
    '000.000.0000',
    STR_TO_DATE('01-JAN-2020', '%d-%M-%Y'), 'IT_PROG', 4000, NULL, NULL,
    60); -- OK
INSERT INTO employee VALUES (903, 'Test', 'Mail2', 'ungueltige-mail-
    adresse', '000.000.0000',
    STR_TO_DATE('01-JAN-2020', '%d-%M-%Y'), 'IT_PROG', 4000, NULL, NULL,
    60); -- Fehler: kein @-Muster
```

Aufgabe 7 SELECT Statement – Grundlagen

Im Rahmen dieser Übung sollen Informationen aus dem `hr`-Schema ausgegeben werden.

7.1 einfaches SELECT-Statement

Erstellen Sie ein `SELECT`-Statement, um die Namen aller Länder auszugeben.

Lösung

```
SELECT country_name FROM hr.country;
```

7.2 aktuelle Uhrzeit

Erstellen Sie ein `SELECT`-Statement, um die aktuelle Uhrzeit in der Form “HH:MM” auszugeben.

Lösung

```
SELECT NOW(), DATE_FORMAT(NOW(), '%H:%i') FROM DUAL;
```

7.3 DISTINCT

Erstellen Sie ein `SELECT`-Statement, welches duplikatfrei alle Manager-IDs liefert.

Lösung

```
SELECT DISTINCT manager_id FROM hr.employee WHERE manager_id IS NOT NULL;
```

7.4 String-Operationen

Erstellen Sie ein `SELECT`-Statement, um die Adressinformationen aller Locations auszugeben. Die Spalte “Stadt” soll das Format `<country_id> - <postal_code> <city> (<state_province>)` haben; statt `NULL` soll “unbekannt” ausgegeben werden.

Lösung

```
SELECT
    street_address AS Adresse,
    IFNULL(
        CONCAT(country_id, ' - ', postal_code, ' ', city, ' (',
            state_province, ')'),
        'unbekannt'
    ) AS Stadt
FROM hr.location;
```

7.5 Filtern

Erstellen Sie ein `SELECT`-Statement, um alle Informationen des CEO (= Mitarbeiter ohne Vorgesetzten) auszugeben.

Lösung

```
SELECT *
FROM hr.employee
WHERE manager_id IS NULL;
```

7.6 Filtern nach mehreren Kriterien

Erstellen Sie ein SELECT-Statement, um aus der “job”-Relation alle Jobbeschreibungen auszugeben, deren Mindestgehalt > 2000 ist und deren Bezeichnung “Clerk” enthält.

Lösung

```
SELECT *
FROM hr.job
WHERE min_salary > 2000 AND LOWER(job_id) LIKE '%clerk%';
```

7.7 Subqueries

Erstellen Sie ein SELECT-Statement, um die vollständigen Informationen aller Mitarbeiter auszugeben, die eine Managerposition innehaben.

Lösung

```
SELECT *
FROM hr.employee
WHERE employee_id IN (
    SELECT DISTINCT manager_id FROM hr.employee WHERE manager_id IS NOT
    NULL
);
```

7.8 komplexes SQL-Statement

Erstellen Sie ein SELECT-Statement zur Ausgabe von Name, E-Mail, Telefon, Einstellungsdatum (inkl. auf 1 Stelle gerundeter Betriebszugehörigkeit in Jahren zum 01.01.2023), Gehalt (in kEUR) und Vorgesetztem (bzw. “niemand”) aller Manager mit Gehalt > 10000, die vor dem 01.01.1996 eingestellt wurden.

Lösung

Das Ergebnis muss **exakt** der vorgegebenen Formatierung entsprechen (Groß-/Kleinschreibung, Abkürzungen, Attributzusammenfassung, NULL-Behandlung):

```
SELECT
    CONCAT(first_name, ' ', last_name) AS Name,
    REPLACE(LOWER(email), '@example.com', '@...') AS EMail,
    SUBSTR(phone_number, 5) AS Telefon,
    CONCAT(
        DATE_FORMAT(hire_date, '%d.%m.%Y'), ' (',
        ROUND(DATEDIFF(STR_TO_DATE('01.01.2023', '%d.%m.%Y'), hire_date)
            / 365, 1),
        ' Jahre)'
    ) AS Einstellungsdatum,
    CONCAT(ROUND(salary/1000, 0), 'k EUR') AS Gehalt,
    IFNULL(manager_id, 'niemand') AS Vorgesetzter
```

```
FROM hr.employee
WHERE
    salary > 10000 AND
    hire_date < STR_TO_DATE('1996-01-01', '%Y-%m-%d') AND
    employee_id IN (SELECT DISTINCT manager_id FROM hr.employee WHERE
        manager_id IS NOT NULL);
```

Aufgabe 8 SELECT Statement – Teil 2

8.1 Sortieren von Tupeln

8.1.1 Telefonbuch

Alphabetisch aufsteigend nach Nach- und Vornamen sortierte Liste (Nachname, Vorname, Telefonnummer) aller Mitarbeiter.

Lösung

```
SELECT last_name, first_name, phone_number
FROM hr.employee
ORDER BY last_name ASC, first_name ASC;
```

8.1.2 Telefonbuch zur Rückwärtssuche

Aufsteigend nach Telefonnummer sortierte Liste (Telefonnummer, Nachname, Vorname).

Lösung

```
SELECT phone_number, last_name, first_name
FROM hr.employee
ORDER BY phone_number ASC;
```

8.2 Gruppieren von Tupeln

8.2.1 Anzahl neuer Mitarbeiter nach Einstellungsjahr

Lösung

```
SELECT
    DATE_FORMAT(hire_date, '%Y') AS Einstellungsjahr,
    COUNT(employee_id) AS Mitarbeiterzahl
FROM hr.employee
GROUP BY Einstellungsjahr;
```

8.2.2 Anzahl der Standorte pro Land

Absteigend nach Anzahl der Standorte sortiert.

Lösung

```
SELECT
    country_id AS Laendercode,
    COUNT(location_id) AS Standortanzahl
FROM hr.location
GROUP BY country_id
ORDER BY Standortanzahl DESC;
```

8.3 Verknüpfung von Relationen

8.3.1 Abteilungsdetails

Liste aller Abteilungen mit Region, Land, Bundesland, Adresse (PLZ, Ort, Anschrift) und Abteilungsname.

Lösung

```

SELECT
    region.region_name ,
    country.country_name ,
    location.state_province ,
    CONCAT(location.postal_code , ' ' , location.city , ' ' , location.
        street_address) AS Adresse ,
    department.department_name
FROM hr.department
JOIN hr.location ON department.location_id = location.location_id
JOIN hr.country ON location.country_id = country.country_id
JOIN hr.region ON country.region_id = region.region_id;

```

8.3.2 Zuordnung von Mitarbeitern zu Projekten

Liste mit Projektname, Vor-/Nachname des Mitarbeiters und Anteil der Arbeitszeit.

Lösung

```

SELECT
    project.project_name ,
    employee.first_name ,
    employee.last_name ,
    employee2project.assoc_pct
FROM hr.project
INNER JOIN hr.employee2project ON project.project_id =
    employee2project.project_id
INNER JOIN hr.employee ON employee.employee_id = employee2project.
    employee_id;

```

8.4 Gruppieren von Tupeln aus verknüpften Tabellen**8.4.1 Überlastete Mitarbeiter**

Liste der Mitarbeiter, die zu mehr als 100% zu Projekten zugeordnet sind (Vor-/Nachname, Gesamtauslastung, Komma-separierte Projektliste).

Lösung

```

SELECT
    employee.last_name AS Nachname ,
    employee.first_name AS Vorname ,
    SUM(employee2project.assoc_pct) AS Gesamtauslastung ,
    GROUP_CONCAT(project.project_name SEPARATOR ', ') AS Projekte
FROM hr.project
INNER JOIN hr.employee2project ON project.project_id =
    employee2project.project_id
INNER JOIN hr.employee ON employee.employee_id = employee2project.
    employee_id
GROUP BY employee.first_name , employee.last_name
HAVING Gesamtauslastung > 1;

```

Diese Abfrage liefert bei den Beispieldaten aus Aufgabe 6.1 genau die Zeile Baer | Hermann | 1.10 | Europe Sales Campaign, US Sales Campaign – da die Constraint aus Aufgabe 5.3 (Summe der Auslastung $\leq 100\%$) bewusst *nicht* als harter CHECK umgesetzt wurde, konnte dieser Datensatz überhaupt erst entstehen.

Aufgabe 9 Mengenoperationen und Views

9.1 Mengenoperationen

Anlässlich eines Firmenjubiläums soll den drei 1996 oder früher eingestellten Mitarbeitern mit dem niedrigsten Gehalt *sowie* den drei Mitarbeitern mit dem insgesamt niedrigsten Gehalt (unabhängig vom Einstelldatum) ein Bonus gezahlt werden. Erstellen Sie ein SQL-Statement, das diesen – nach aufsteigendem Gehalt sortierten – Personenkreis ermittelt.

Lösung

```
(SELECT employee_id, first_name, last_name, hire_date, job_id, salary
FROM hr.employee
WHERE DATE_FORMAT(hire_date, '%Y') <= 1996
ORDER BY salary ASC LIMIT 3)
UNION
(SELECT employee_id, first_name, last_name, hire_date, job_id, salary
FROM hr.employee
ORDER BY salary ASC LIMIT 3)
ORDER BY salary ASC;
```

UNION entfernt automatisch Duplikate (z. B. Mitarbeiter, die in beiden Teilmengen vorkommen), sodass am Ende höchstens 6, ggf. aber auch weniger Zeilen ausgegeben werden.

9.2 Views

Erstellen Sie eine View `employee_details` mit den Attributen `employee_id`, `job_id`, `manager_id`, `department_id`, `location_id`, `country_id`, `first_name`, `last_name`, `salary`, `commission_pct`, `department_name`, `job_title`, `city`, `state_province`, `country_name`, `region_name`. Fragen Sie anschließend möglichst einfach alle Details über die View ab.

Lösung

```
CREATE OR REPLACE VIEW employee_details AS
SELECT
    employee_id, job_id, manager_id, department_id, location_id,
    country_id,
    first_name, last_name, salary, commission_pct,
    department_name, job_title, city, state_province, country_name,
    region_name
FROM hr.employee
JOIN hr.department USING (department_id)
JOIN hr.job USING (job_id)
JOIN hr.location USING (location_id)
JOIN hr.country USING (country_id)
JOIN hr.region USING (region_id);

-- lesender Zugriff:
SELECT * FROM employee_details;
```

Die Verwendung von `USING(...)` ist hier möglich, weil die verknüpfenden Fremdschlüsselspalten in beiden beteiligten Tabellen jeweils identisch benannt sind (`department_id`, `job_id`, `location_id`, `country_id`, `region_id`); `USING` vermeidet dabei, dass die gleichnamige Spalte doppelt im Ergebnis erscheint.